

1.

```
/**
 * Returns the number of digits of {@code n}.
 *
 * @param n
 *         {@code NaturalNumber} whose digits to count
 * @return the number of digits of {@code n}
 * @ensures numberOfDigits = [number of digits of n]
 */
private static int numberOfDigits(NaturalNumber n) {
    NaturalNumber temp = new NaturalNumber2(n);
    Int count = 0;
    If(!temp.isZero()){
        Temp.divideBy10();
        Count = 1 + numberOfDigits(temp);
    }
    Return count;
}
```

2.

```
/**
 * Returns the sum of the digits of {@code n}.
 *
 * @param n
 *         {@code NaturalNumber} whose digits to add
 * @return the sum of the digits of {@code n}
 * @ensures sumOfDigits = [sum of the digits of n]
 */
private static int sumOfDigits(NaturalNumber n) {
    NaturalNumber temp = new NaturalNumber2(n);
    Int sum = temp.divideBy10();
    If(!temp.isZero()){
        Sum = sum + sumOfDigits(temp);
    }
    Return sum;
}
```

3.

```
/**
 * Returns the sum of the digits of {@code n}.
 *
 * @param n
 *         {@code NaturalNumber} whose digits to add
 * @return the sum of the digits of {@code n}
```

```

    * @ensures sumOfDigits = [sum of the digits of n]
    */
    private static NaturalNumber sumOfDigits(NaturalNumber n) {
        NaturalNumber temp = new NaturalNumber2(n);
        NaturalNumber sum = new NaturalNumber2(temp.divideBy10());
        If(!temp.isZero()){
            Sum.add(sumOfDigits(temp));
        }
        Return sum;
    }
}

```

4.

```

/**
 * Divides {@code n} by 2.
 *
 * @param n
 *          {@code NaturalNumber} to be divided
 * @updates n
 * @ensures 2 * n <= #n < 2 * (n + 1)
 */

```

```

private static void divideBy2(NaturalNumber n) {
    if(n.isZero()){
        return;
    }
    NaturalNumber copy = n.newInstance();
    Copy.copyFrom(n);
    Int lastDigit = n.divideBy10();
    divideBy2(n);
    copy.divideBy10();
    NaturalNumber double = n.newInstance();
    doubleN.copyFrom(n);
    NaturalNumber two = new NaturalNumber(2);
    doubleN.multiply(two);
    NaturalNumber diff = copy.newInstance();
    Diff.copyFrom(copy);
    Diff.subtract(doubleN);
    Int carry = 0;
    If(!diff.isZero()){
        Carry = 1;
    }
    Int x = carry * 10 + lastDigit;
    Int new digit = x/2;
    n.multiplyBy10(newDigit);
}

```

5.

```
/**
```

```
 * Divides {@code n} by 2.
```

```
 *
```

```
 * @param n
```

```
 *           {@code NaturalNumber} to be divided
```

```
 * @updates n
```

```
 * @ensures 2 * n <= #n < 2 * (n + 1)
```

```
 */
```

```
private static void divideBy2(NaturalNumber n) {...}
```